

Standing Up an Apache Web Server EC2 Instance and Sending Logs to Amazon CloudWatch

Christopher Samugana

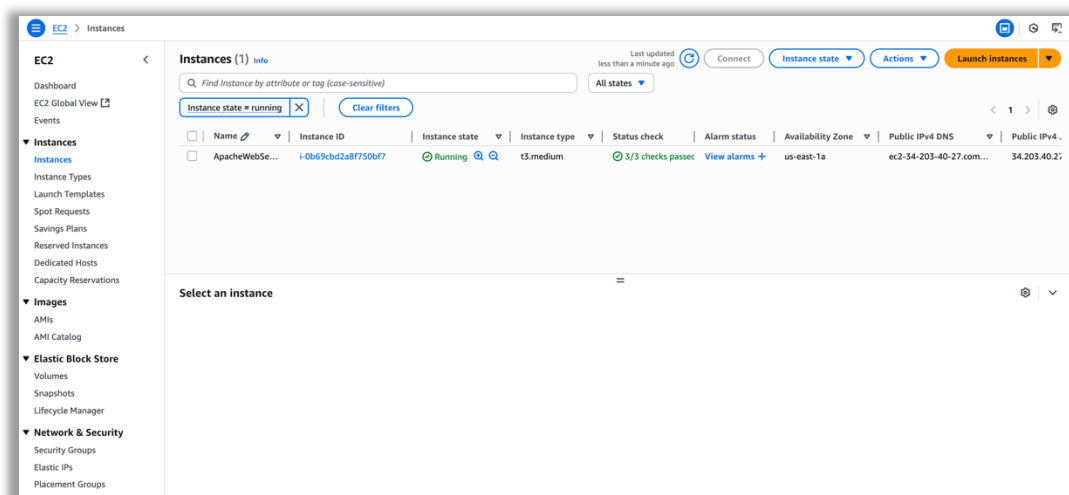
Introduction

In this project, I configured an Amazon EC2 instance to forward custom log files to Amazon CloudWatch using the CloudWatch Agent. After confirming that the log delivery was successful, I implemented a monitoring and alerting system using Metric Filters and Amazon SNS to catch specific warning and error messages, particularly HTTP 404 responses.

A. Installed and Configured the Amazon CloudWatch Agent

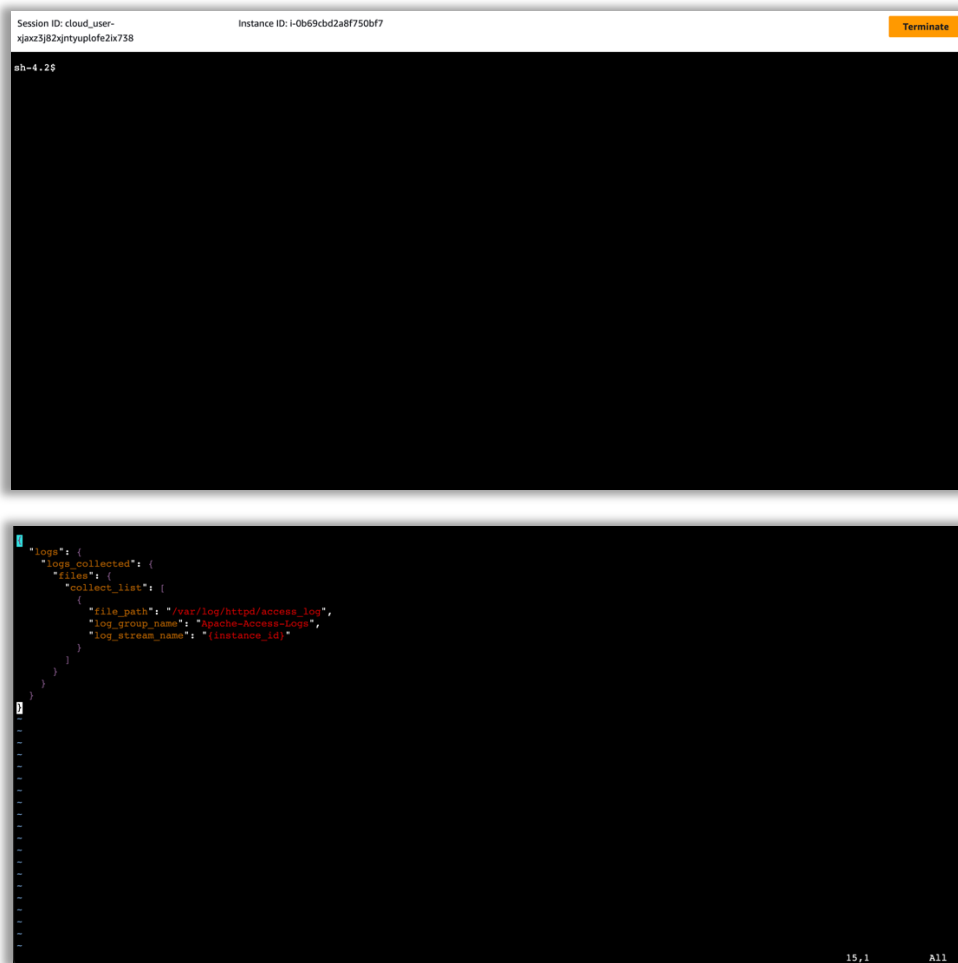
I began by logging into the AWS Management Console with the lab-provided credentials, ensuring I was working within the us-east-1 region.

From the EC2 dashboard, I located the ApacheWebServer instance. I copied the Public IP address and opened it in a new browser tab, changing the protocol from HTTPS to HTTP to verify that the Hello Gurus! page was accessible. I kept this tab open for later use.

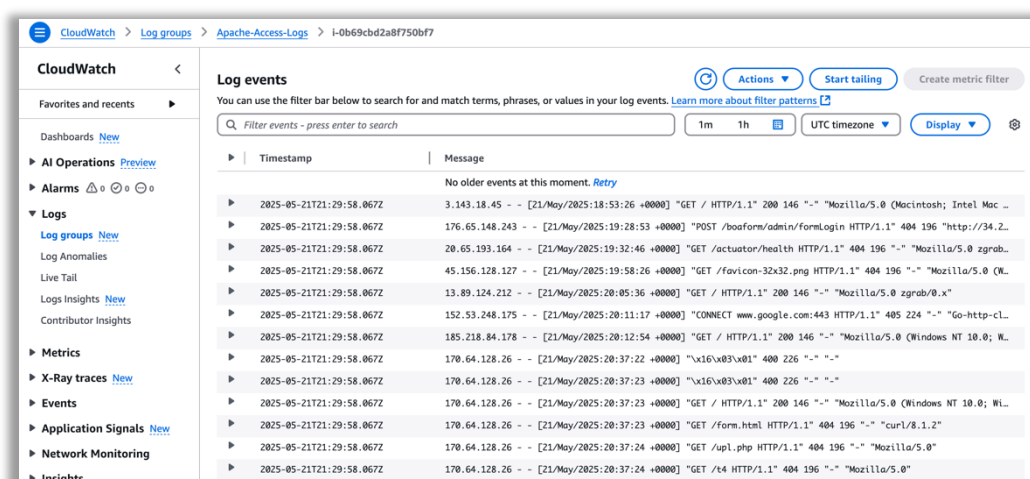


Back in the AWS Console, I connected to the instance using Session Manager under the Connect options.

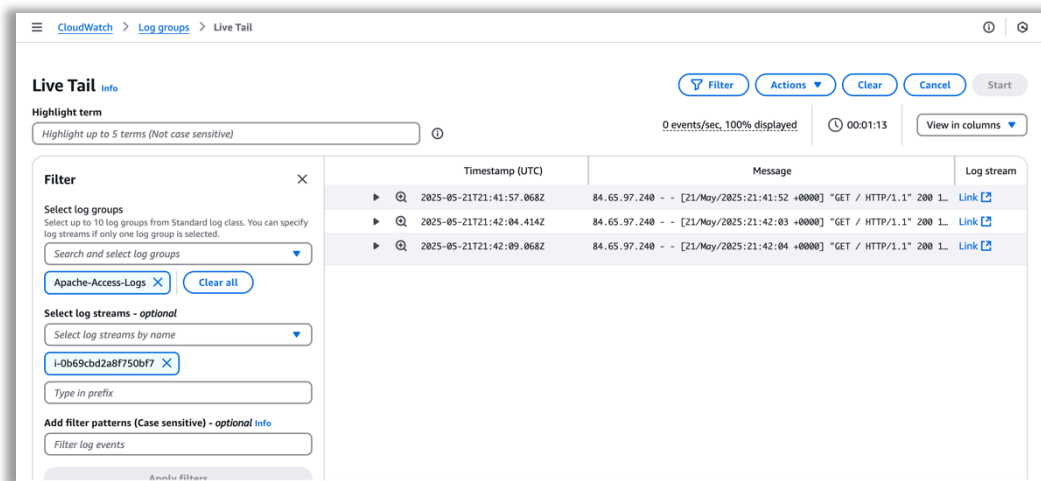
Inside the terminal session, I installed the Amazon CloudWatch Agent. I then created a configuration file that specified which log files the agent should collect, where to send them, and how to structure the log groups and streams in CloudWatch.



After saving the configuration file, I applied it using the CloudWatch Agent control command. Then, I refreshed the web server page multiple times to generate some initial log entries.

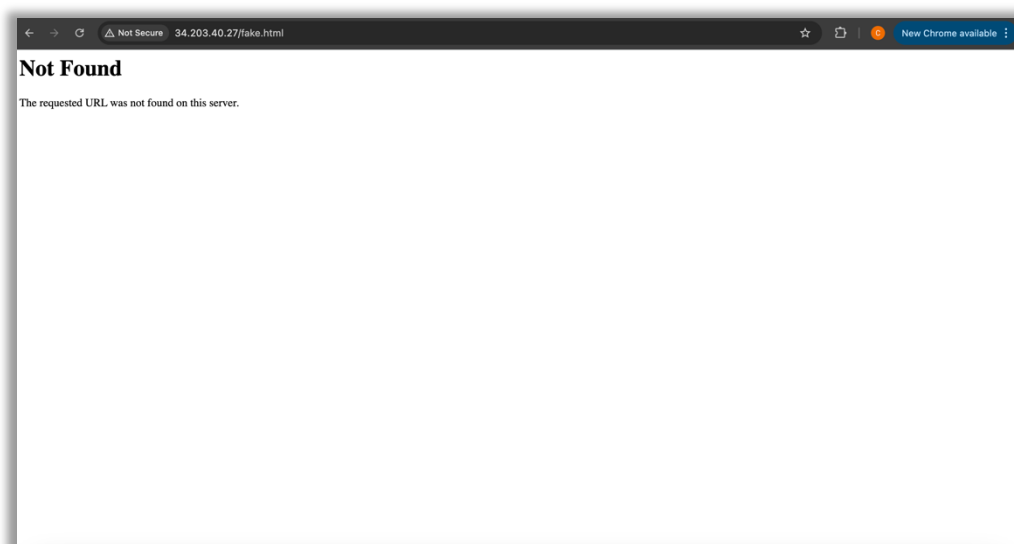


In Amazon CloudWatch, I navigated to Log Groups, found Apache-Access-Logs, and opened the corresponding Log Stream. I initiated the Start tailing function and confirmed that logs were flowing in as expected while refreshing the web server page.



B. Created a Metric Filter for 404 Status Codes

To create a custom metric filter for 404 errors, I first visited a non-existent page on the web server to generate a 404 response.



I returned to the **Apache-Access-Logs** log group in CloudWatch, confirmed that the 404 entry appeared, and exited the tailing session.



Under the **Metric filters** tab, I selected **Create metric filter** and defined a filter pattern that captured 404 status codes. I tested the filter against recent log data and confirmed that it matched the expected entries.

Create filter pattern

You can use metric filters to monitor events in a log group as they are sent to CloudWatch Logs. You can monitor and count specific terms or extract values from log events and associate the results with a metric. [Learn more about pattern syntax.](#)

Filter pattern

Specify the terms or pattern to match in your log events to create metrics.

☐ Enable metric filter on transformed logs
When enabled, metric filter will be applied to transformed logs. When disabled, metric filter will be applied to original logs.

Test pattern

Select log data to test

Log event messages

Type log data to test with your Filter Pattern. Please use line breaks to separate log events.

```
180.178.94.73 - - [21/May/2025:21:15:46 +0000] "GET /vendor/phpunit/Util/PHP/eval-stdin.php HTTP/1.1" 404 196 "-" "Custom-AsyncHttpClient"
180.178.94.73 - - [21/May/2025:21:15:46 +0000] "GET /vendor/phpunit/src/Util/PHP/eval-stdin.php HTTP/1.1" 404 196 "-" "Custom-AsyncHttpClient"
180.178.94.73 - - [21/May/2025:21:15:47 +0000] "GET /vendor/phpunit/Util/PHP/eval-
```

[Test pattern](#)

Results

Please select log event messages above and click "Test pattern" to see results.

Test pattern results

Found 41 matches out of 10 events in the sample log.

Event number	Event message
1	180.178.94.73 - - [21/May/2025:21:15:46 +0000] "GET /vendor/phpunit/Util/PHP/eval-stdin.php HTTP/1.1" 404 196 "-" "Custom-AsyncHttpClient"
2	180.178.94.73 - - [21/May/2025:21:15:46 +0000] "GET /vendor/phpunit/src/Util/PHP/eval-stdin.php HTTP/1.1" 404 196 "-" "Custom-AsyncHttpClient"

I named the filter 404 and configured it to publish to a custom namespace called Apache with a metric name of 404. I set the metric value to 1 and selected the unit type as **Count**. After reviewing the setup, I created the metric filter. I then refreshed the fake web page again to generate more 404 errors and verified that the custom namespace and metric appeared under **All metrics** in CloudWatch.

Assign metric

Create filter name

Log events that match the pattern you define are recorded to the metric that you specify. You can graph the metric and set alarms to notify you.

Filter name

Filter pattern

☐ Enable metric filter on transformed logs
When enabled, metric filter will be applied to transformed logs. When disabled, metric filter will be applied to original logs.

Metric details

Metric namespace

Namespaces let you group similar metrics. [Learn more](#)

[Create new](#)

Metric name

Metric name identifies this metric, and must be unique within the namespace. [Learn more](#)

Metric value

Metric value is the value published to the metric name when a Filter Pattern match occurs.

Valid metric values are floating point number (1, 99.9, etc.), numeric field identifiers (\$1, \$2, etc.), or named field identifiers (e.g. \$requestSize for delimited filter pattern or \$status for JSON-based filter pattern - dollar (\$) or dollar dot (\$) followed by alphanumeric and/or underscore (_) characters).

Metric filters (1)

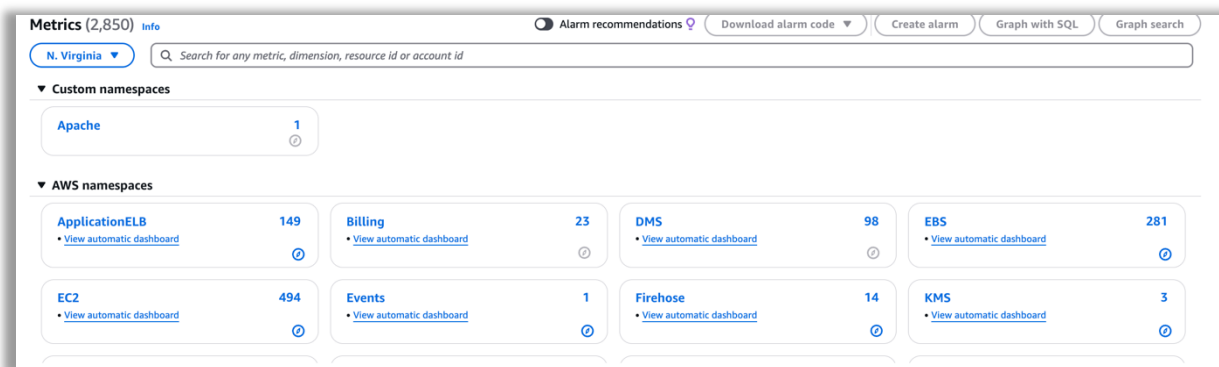
[Edit](#) [Delete](#) [Create alarm](#) [Create metric filter](#)

Filter pattern	Metric	Metric value	Default value	Applied on transformed logs	Unit	Dimensions	Alarms
%b404b%	Apache / 404	1	-	-	Count	-	None

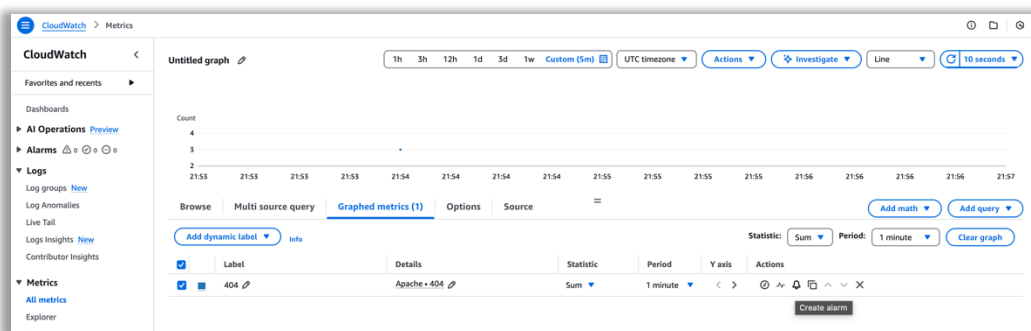
C. Created an SNS Topic and CloudWatch Alarm

With the 404 metric in place, I created a CloudWatch Alarm to send an alert whenever a 404 error occurred.

Within the Apache namespace under All Metrics, I selected the 404 metric. I customized the graph to show data from the last five minutes, auto-refresh every 10 seconds, and use a one-minute period with a Sum statistic.

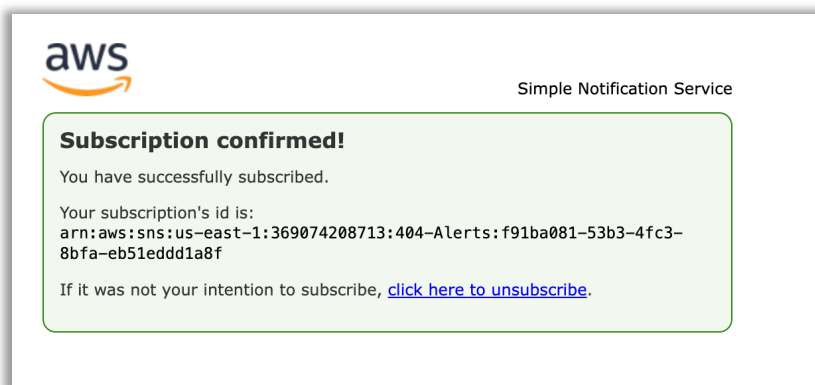
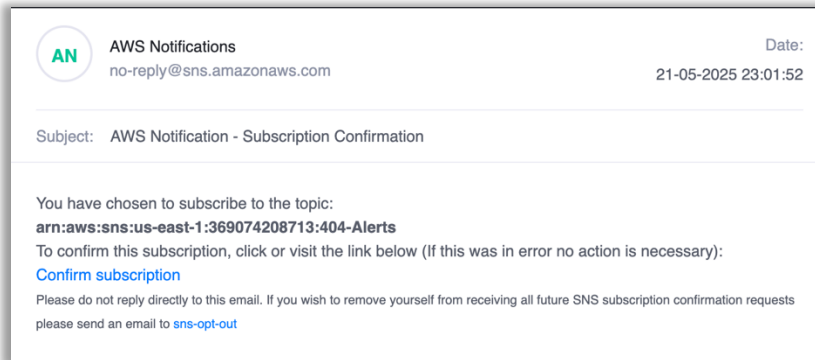


Using the bell icon in the graph's Actions column, I began creating the alarm. I set the condition to trigger when the metric was greater than zero.

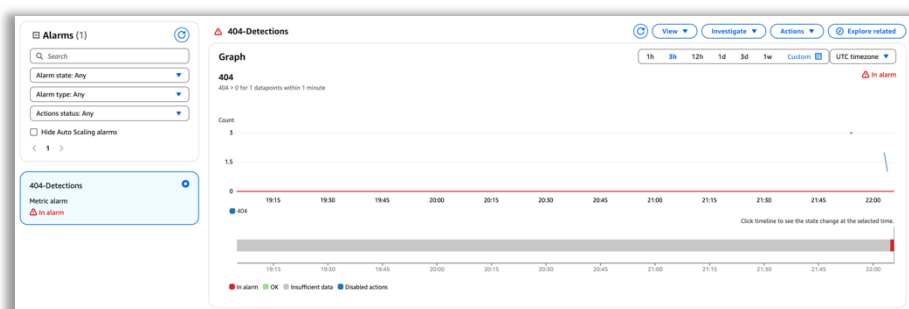
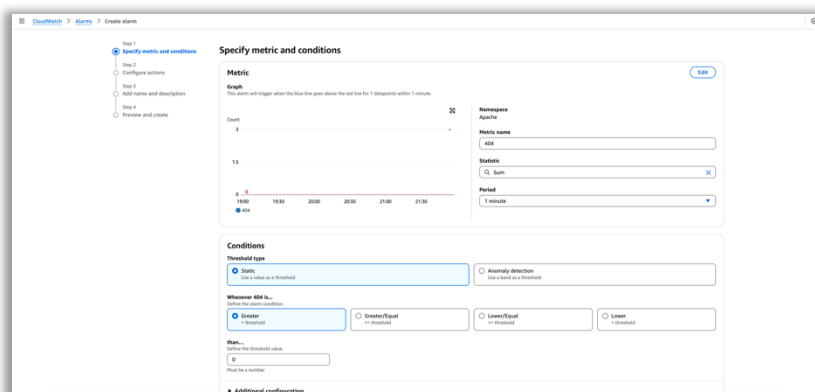


The screenshot shows the 'Configure actions' dialog in the CloudWatch console. It has a sidebar with steps: 'Step 1: Specify metric and conditions', 'Step 2: Configure actions', 'Step 3: Add name and description', and 'Step 4: Preview and create'. The 'Configure actions' step is selected. The main area is titled 'Configure actions' and has a 'Notification' section. Under 'Notification', there is a 'Notification' section with a 'Remove' button. Below this is an 'Alarm state trigger' section with three radio buttons: 'In alarm' (selected), 'OK', and 'Insufficient data'. The 'In alarm' option has a description: 'The metric or expression is outside of the defined threshold.' Below this is a 'Send a notification to the following SNS topic' section with three radio buttons: 'Select an existing SNS topic', 'Create new topic' (selected), and 'Use topic ARN to notify other accounts'. Below this is a 'Create a new topic...' section with a text input field containing '404-Alerts'. Below this is an 'Email endpoints that will receive the notification...' section with a text input field containing 'kobace644@leabro.com'. At the bottom are 'Create topic' and 'Add notification' buttons.

For notifications, I created a new SNS Topic named 404-Alerts and entered an email address to receive alerts. After completing the alarm setup, I confirmed the email subscription via a link sent to the provided inbox.



I named the alarm 404-Detections, reviewed the configuration, and finalized its creation. To verify the system, I triggered more 404 errors by visiting the fake page again.



After a few minutes, the alarm moved from an Insufficient data state to In alarm, and I received an alert email from SNS as expected.

